[알고리즘 과제] 그래프 관련 알고리즘 프로그래밍 - 6주차과제 보고서

- **학번:2371214**
- **이름:이지형**
- **트랙:모바일소프트웨어 & 빅데이터트랙**

1. 과제 개요

해당과제에서는 파일입출력을 통해 그래프 데이터를 동적으로 로드하고, 기존에 교재에서 학습했던 최소 비용신장트리 뿐만 아니라, 최대비용 신장트리 알고리즘(Kruskal,prim)과 최단 경로 알고리즘(Dijkstra, Floyd)를 구현하고 경로상의 위치하는 정점들도 같이 출력하도록 구현하였습니다.

과제에서 제시된 가변 노드 조건에 맞추어 모든 주요 배열은 동적 메모리 할당방식으로 처리하였으며, 프로그램 실행과정에서 메모리 누수가 발생하지 않도록 과제를 작성하였습니다.

2. 주요 요구사항 준수과정

- 내부 배열 연산은 0번을 기준으로 처리하되, 화면 출력 및 사용자 입력(출발 노드와 목적지 노드)은 1번 인덱스를 기준으로 변환시켜서 교재와 기본코드의 표준과 사용자의 편의성을 맞추었습니다.

- 노드개수 및 간선 개수를 파일 읽기 시점에 파악하여 malloc으로 필요한 크기만큼 메모리를 동적 할당하고, 각 알고리즘이 종료시 free로 메모리를 해제하였습니다.

3. 알고리즘 별 구현 내용

-Kruskal 알고리즘 (최소/최대 신장 트리)

- **알고리즘 개요 및 핵심 논리:**
    - 그래프 내의 모든 간선을 가중치 기준으로 정렬한 후, 가중치가 가장 작거나 큰 간선부터 순차적으로 선택하여 신장 트리를 구축하는 탐욕적 알고리즘.
    - 간선 선택 시 사이클이 형성되는지 여부를 판단하기 위해 Union-Find 자료구조를 활용.
- **주요 구현 방식:**
    1. 동적 메모리 할당: 정점의 부모 노드를 저장하는 parent 배열을 (g->n + 1) 크기만큼 malloc으로 동적 할당하고, set_init() 함수를 통해 모두 -1로 초기화.
    2. 간선 정렬: 원본 간선의 손상을 방지하기 위해 g->n_e 크기의 temp_edges 배열을 동적 할당하여 간선 정보를 복사. 표준 라이브러리의 qsort() 함수를 사용하며, st_kind 매개변수 값에 따라 비교 함수를 다르게 적용하여 오름차순 또는 내림차순 정렬을 수행.
    3. 사이클 검사 및 집합 합치기: 정렬된 간선을 순회하며 set_find()로 두 정점의 루트 노드를 확인. 서로 다른 집합에 속해 있으면 해당 간선을 신장 트리에 포함하고 set_union()을 호출하여 두 집합을 합침.
    4. 종료 조건 및 메모리 해제: 선택된 간선의 수가 (n - 1)개가 되면 반복을 즉시 종료하고 총 가중치 합을 출력한 뒤, temp_edges와 parent 메모리를 free()로 해제.

-Prim 알고리즘 (최소/최대 신장 트리)

- **알고리즘 개요 및 핵심 논리:**
    - 하나의 시작 정점에서 출발하여, 이미 선택된 정점들의 집합에 인접한 정점들 중 가중치가 가장 작거나 큰 간선으로 연결된 정점을 하나씩 추가해 나가는 알고리즘.
- **주요 구현 방식:**
    1. 동적 메모리 할당 및 초기화: 정점 선택 여부를 기록하는 selected, 신장 트리 집합에서 각 정점까지의 최단 가중치를 저장하는 dist, 최소 가중치

간선으로 연결된 인접 정점을 기록하는 near_node 배열을 각각 g->n 크기만큼 malloc으로 동적 할당.

2. 가중치 갱신: 새로 선택된 정점 target을 트리에 추가한 후, 선택되지 않은 다른 정점 v들에 대해 target을 거쳐서 연결되는 간선의 가중치가 기존 dist[v]보다 우수한 경우 dist[v]와 near_node[v]를 갱신함.

-Dijkstra 알고리즘 (단일 출발점 최단 경로)

- **알고리즘 개요 및 핵심 논리:**
  - 음의 가중치가 없는 그래프에서 특정 출발 정점으로부터 다른 모든 정점까지의 최단 경로를 구하는 알고리즘
- **주요 구현 방식:**
  1. 최단 거리 정점 확정: dist[start] = 0, found[start] = TRUE로 설정한 뒤, 매 단계마다 아직 방문하지 않은 정점 중 dist 값이 가장 작은 정점 u를 선택하여 found[u] = TRUE로 최단 거리를 확정합니다.
  2. 경로 갱신: 정점 u를 거쳐서 정점 w로 가는 거리가 기존 dist[w]보다 짧은 경우(dist[u] + g->adj_mat[u][w] < dist[w]), dist[w]를 갱신하고 prev[w] = u로 직전 방문 정점을 기록.
  3. 경로 추적 출력: 최단 거리 출력 시, print_dijkstra_path() 재귀 함수를 호출하여 prev 배열을 역추적함으로써 출발 노드 -> ... -> 목적지 노드 형태의 전체 이동 경로를 순서대로 출력함.

-Floyd 알고리즘 (모든 정점 쌍 최단 경로)

- **알고리즘 개요 및 핵심 논리:**
  - 그래프 내의 모든 정점 쌍 (u, v) 간의 최단 거리를 구하는 알고리즘.
    정점 k를 경유지로 설정하여 거리를 단계적으로 갱신.
- **주요 구현 방식:**

1. 3중 루프를 통한 최단 경로 계산:
    - 경유 정점 k, 출발 정점 i, 도착 정점 j에 대해 3중 for문을 수행함.
    - 조건식 A[i][k] + A[k][j] < A[i][j]를 만족하면 더 짧은 경로로 판단하여 A[i][j] 값을 갱신하고, 경유지 정점 k를 next_node[i][j] = k로 기록.
2. 경유지 역추적 및 메모리 완전 해제:
    - print_floyd_path() 재귀 함수를 이용해 next_node에 기록된 경유지를 분할 정복 방식으로 탐색하여 시작 정점부터 도착 정점까지의 전체 경로를 출력함.

4.결론 및 종합 소감

-이번 6주차 과제를 시행해보면서 4개의 최대-최소 비용신장트리, 최단경로 알고리즘을 직접 구현해볼수 있었고, 교재 학습시에는 동적 메모리 할당방식없이 알고리즘을 구현하였는데, 이번과제에서 동적메모리 할당방식으로 처리하는 알고리즘을 작성해볼수 있어 좋은 학습 기회가 된 것 같습니다. 또한 각 수업마다 마지막에 진행하셨던 추가적인 코드와 함수를 직접 코드안에 녹여넣는 작업을 하며, 수업때 배운 내용을 체화할수 있었습니다. 내부 연산 시 사용되는 0-based 인덱스와 사용자 입력 및 화면 출력 시의 1-based 인덱스 간의 변환을 매끄럽게 처리하는 것이 어려웠던 것 같습니다.

5.실행결과 화면 캡쳐

Microsoft Visual Studio 디버그

```
그래프 인접행렬 표현:
  0   8  10 999 999 999 999 999
  8   0 999   9  11 999 999 999
 10 999   0   5 999 999 999  15
999   9   5   0  13  12 999  14
999  11 999  13   0  11  10 999
999 999 999  12  11   0   7  13
999 999 999 999  10   7   0 999
999 999  15  14 999  13 999   0


크루스칼 신장 트리 알고리즘
최소(0을 입력) 또는 최대(1을 입력) 신장 트리? 1

=== [최대비용신장트리 Kruskal 알고리즘 결과] ===
(8, 3, 15)
(8, 4, 14)
(5, 4, 13)
(8, 6, 13)
(5, 2, 11)
(7, 5, 10)
(3, 1, 10)
총 가중치 합: 86

프림 신장 트리 알고리즘
최소(0을 입력) 또는 최대(1을 입력) 신장 트리? 0
프림 알고리즘을 위한 출발 노드 번호? 2

=== [최소비용신장트리 Prim 알고리즘 결과 (출발 노드: 2)] ===
(2, 1, 8)
(2, 4, 9)
(4, 3, 5)
(2, 5, 11)
(5, 7, 10)
(7, 6, 7)
(6, 8, 13)
총 가중치 합: 63

Dijkstra 최단 경로 알고리즘을 위한 출발 노드 번호? 3

=== [Dijkstra 최단 경로 (출발 노드: 3)] ===
목적지 노드 1 | 최단 거리:  10 | 경로: 3 -> 1
목적지 노드 2 | 최단 거리:  14 | 경로: 3 -> 4 -> 2
목적지 노드 4 | 최단 거리:   5 | 경로: 3 -> 4
목적지 노드 5 | 최단 거리:  18 | 경로: 3 -> 4 -> 5
목적지 노드 6 | 최단 거리:  17 | 경로: 3 -> 4 -> 6
목적지 노드 7 | 최단 거리:  24 | 경로: 3 -> 4 -> 6 -> 7
목적지 노드 8 | 최단 거리:  15 | 경로: 3 -> 8

Floyd 최단 경로 알고리즘을 위한 출발 노드 번호와 도착 노드 번호? 2 6

=== [Floyd 특정 노드 쌍 최단 경로 (2 -> 6)] ===
최단 거리: 21 | 경로: 2 -> 4 -> 6

=== [Floyd 모든 노드 쌍 최단 경로 결과] ===
1 -> 2 | 거리:   8 | 경로: 1 -> 2
1 -> 3 | 거리:  10 | 경로: 1 -> 3
1 -> 4 | 거리:  15 | 경로: 1 -> 3 -> 4
1 -> 5 | 거리:  19 | 경로: 1 -> 2 -> 5
1 -> 6 | 거리:  27 | 경로: 1 -> 3 -> 4 -> 6
1 -> 7 | 거리:  29 | 경로: 1 -> 2 -> 5 -> 7
1 -> 8 | 거리:  25 | 경로: 1 -> 3 -> 8
2 -> 1 | 거리:   8 | 경로: 2 -> 1
2 -> 3 | 거리:  14 | 경로: 2 -> 4 -> 3
2 -> 4 | 거리:   9 | 경로: 2 -> 4
2 -> 5 | 거리:  11 | 경로: 2 -> 5
2 -> 6 | 거리:  21 | 경로: 2 -> 4 -> 6
2 -> 7 | 거리:  21 | 경로: 2 -> 5 -> 7
2 -> 8 | 거리:  23 | 경로: 2 -> 4 -> 8
3 -> 1 | 거리:  10 | 경로: 3 -> 1
3 -> 2 | 거리:  14 | 경로: 3 -> 4 -> 2
3 -> 4 | 거리:   5 | 경로: 3 -> 4
3 -> 5 | 거리:  18 | 경로: 3 -> 4 -> 5
3 -> 6 | 거리:  17 | 경로: 3 -> 4 -> 6
3 -> 7 | 거리:  24 | 경로: 3 -> 4 -> 6 -> 7
3 -> 8 | 거리:  15 | 경로: 3 -> 8
4 -> 1 | 거리:  15 | 경로: 4 -> 3 -> 1
4 -> 2 | 거리:   9 | 경로: 4 -> 2
4 -> 3 | 거리:   5 | 경로: 4 -> 3
4 -> 5 | 거리:  13 | 경로: 4 -> 5
4 -> 6 | 거리:  12 | 경로: 4 -> 6
4 -> 7 | 거리:  19 | 경로: 4 -> 6 -> 7
4 -> 8 | 거리:  14 | 경로: 4 -> 8
5 -> 1 | 거리:  19 | 경로: 5 -> 2 -> 1
5 -> 2 | 거리:  11 | 경로: 5 -> 2
5 -> 3 | 거리:  18 | 경로: 5 -> 4 -> 3
5 -> 4 | 거리:  13 | 경로: 5 -> 4
5 -> 6 | 거리:  11 | 경로: 5 -> 6
5 -> 7 | 거리:  10 | 경로: 5 -> 7
5 -> 8 | 거리:  24 | 경로: 5 -> 6 -> 8
6 -> 1 | 거리:  27 | 경로: 6 -> 4 -> 3 -> 1
6 -> 2 | 거리:  21 | 경로: 6 -> 4 -> 2
6 -> 3 | 거리:  17 | 경로: 6 -> 4 -> 3
6 -> 4 | 거리:  12 | 경로: 6 -> 4
6 -> 5 | 거리:  11 | 경로: 6 -> 5
6 -> 7 | 거리:   7 | 경로: 6 -> 7
6 -> 8 | 거리:  13 | 경로: 6 -> 8
7 -> 1 | 거리:  29 | 경로: 7 -> 5 -> 2 -> 1
7 -> 2 | 거리:  21 | 경로: 7 -> 5 -> 2
7 -> 3 | 거리:  24 | 경로: 7 -> 6 -> 4 -> 3
7 -> 4 | 거리:  19 | 경로: 7 -> 6 -> 4
7 -> 5 | 거리:  10 | 경로: 7 -> 5
7 -> 6 | 거리:   7 | 경로: 7 -> 6
7 -> 8 | 거리:  20 | 경로: 7 -> 6 -> 8
8 -> 1 | 거리:  25 | 경로: 8 -> 3 -> 1
8 -> 2 | 거리:  23 | 경로: 8 -> 4 -> 2
8 -> 3 | 거리:  15 | 경로: 8 -> 3
8 -> 4 | 거리:  14 | 경로: 8 -> 4
8 -> 5 | 거리:  24 | 경로: 8 -> 6 -> 5
8 -> 6 | 거리:  13 | 경로: 8 -> 6
8 -> 7 | 거리:  20 | 경로: 8 -> 6 -> 7

C:\Users\User\OneDrive\바탕 화면\2학년2학기\알고리즘(자료구조2)\2371214_이지형_6주차과제\x64\Debug\graph_homework.exe(프로세스 34620)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```